

# RAG คืออะไร?

## สอนแบบเห็นภาพ ไม่ต้องเริ่มจากคณิตศาสตร์

จาก "AI ตอบจากความจำ" → "AI เปิดตำราก่อนตอบ" พร้อมภาพเปรียบเทียบ

Embedding, Vector และ Vector Database แบบ GPS

เอกสาร



ค้นความหมาย



LLM ตอบ

# คำว่า **RAG** ย่อมาจากอะไร?

## **R = Retrieval**

“ค้นหา” ข้อมูลที่เกี่ยวข้องกับคำถามจากเอกสารหรือฐานความรู้

## **A = Augmented**

“เติมข้อมูลให้” LLM ก่อนตอบ เหมือนยื่นโปยหรือหน้าหนังสือที่เกี่ยวข้องให้ดู

## **G = Generation**

“สร้างคำตอบ” โดยให้ LLM อ่านข้อมูลที่ค้นหาแล้วเรียบเรียงเป็นภาษาคน

สูตรง่าย: RAG = ค้นหา → เอาข้อมูลมาให้ AI อ่าน → แล้วค่อยตอบ

# RAG เหมือน “เด็กเปิดหนังสือก่อนตอบครู”

## ไม่มี RAG

ครูถาม: “ดาวเคราะห์ดวงไหนใหญ่ที่สุด?”

เด็กตอบจาก **ความจำ** — จำได้ก็ตอบถูก จำผิดก็มีโอกาสตอบผิด

## มี RAG

ครูถามคำถามเดิม

เด็ก **เปิดหนังสือ** → หาหน้าที่เกี่ยวข้อง → อ่าน → ตอบ

**RAG ไม่ได้ทำให้เด็ก “จำเก่งขึ้น” แต่ทำให้เด็ก มีหนังสือให้เปิดก่อนตอบ**

# ตัวอย่าง: AI ตอบคำถามจาก “คู่มือร้านกาแฟ”



คู่มือร้าน



ค้นหัวข้อที่เกี่ยวข้อง



AI อ่าน



ตอบพนักงาน

## พนักงานถาม

“ถ้าลูกค้าขอเปลี่ยนนมเป็น Oat Milk คิดเพิ่ม  
เท่าไร?”

## RAG ทำอะไร

ค้นเฉพาะหน้าที่เขียนเรื่อง **ราคาเครื่องดื่มและตัวเลือกนม** แล้ว  
ส่งส่วนนั้นให้ AI ตอบ

# ตัวอย่าง: พนักงานใหม่ถาม “ลาจิดได้กี่วัน?”

บริษัทมี Employee Handbook 120 หน้า

เราไม่จำเป็นต้องส่ง 120 หน้าให้ AI ทุกครั้ง

RAG จะค้นเฉพาะหัวข้อ **Leave Policy / ลาจิด / วันลา**



**ถาม**

“พนักงานลาจิดได้กี่วัน?”



**ค้น**

หา Chunk ที่พูดเรื่องลาจิด



**ตอบ**

LLM อ่าน Chunk นั้น แล้วตอบพร้อมอ้างอิงเอกสาร

## ตัวอย่าง: คู่มือ 3D Printer

### สถานการณ์

เรามีคู่มือ 200 หน้า และผู้ใช้ถามว่า  
“Nozzle อุดตัน ต้องทำอะไร?”

📖 คู่มือ 200 หน้า



✂️ แบ่งเป็น Chunk



🔍 หาชิ้นที่เกี่ยวข้องกับ Nozzle



🤖 LLM อ่านแล้วตอบ

นี่คือ **RAG**: AI ไม่ต้องจำคู่มือทั้งเล่ม แต่ต้อง “หาเนื้อหาที่ใช้ให้เจอ” ก่อนตอบ

# จบ 30 สไลด์นี้ นักเรียนควรเข้าใจอะไร?

## 1. RAG คืออะไร

อธิบายได้ด้วยคำง่าย ๆ ว่า “ค้นก่อน แล้วค่อยตอบ”

## 2. Embedding / Vector ต่างกันอย่างไร

Embedding = วิธีแปลงความหมายเป็นพิกัด · Vector = พิกัดที่ได้

## 3. Vector Database ทำอะไร

ค้นหาพิกัดที่อยู่ใกล้กัน เหมือนระบบ GPS หา “จุดใกล้เคียง”

## 4. Python ต่อทุกอย่างอย่างไร

เห็น Workflow ตั้งแต่ PDF → Chunk → Search → LLM

# ก่อนรู้จัก RAG ให้เข้าใจปัญหาก่อน

LLM เหมือน **คนเก่งที่อ่านหนังสือมาเยอะ** แต่ไม่ได้ถือเอกสารของ  
บริษัทเราอยู่ในมือ



## ความรู้ทั่วไป

มีเยอะ แต่ไม่ได้ครอบคลุมข้อมูลเฉพาะของเราเสมอ



## ข้อมูลเปลี่ยนได้

คู่มือ ราคา ขึ้นตอน หรือข้อมูลภายในอาจอัปเดต



## อาจเดา

ถ้าไม่มีข้อมูลพอ LLM อาจตอบดูน่าเชื่อถือแต่ผิด



# RAG = ให้ AI เปิดตำราก่อนตอบ

## ไม่มี RAG

ถามเพื่อนจากความจำ

"จำได้ไหมว่าในคู่มือบอกอะไร?"

เพื่อนอาจจำถูก... หรือเดา

## มี RAG

ให้เพื่อนไปเปิดหนังสือก่อน

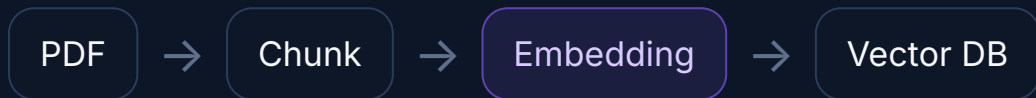
ค้นหน้าที่เกี่ยวข้อง → อ่าน → ค่อยตอบ

คำตอบมี "หลักฐาน" จากเอกสารของเรา

**RAG = Retrieval-Augmented Generation** → "ดึงข้อมูลมาเสริมก่อนสร้างคำตอบ"

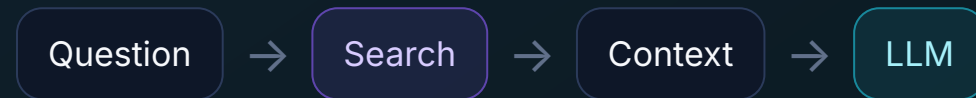
# RAG มี 2 ช่วงใหญ่ ๆ

## A. เตรียมคลังความรู้



ทำเมื่อเพิ่ม/แก้ไขเอกสาร

## B. ตอนผู้ใช้ถาม



เกิดทุกครั้งที่มีการถาม

# เอกสาร = หนังสือในห้องสมุดของเรา



คู่มือเครื่อง เอกสารอบรม SOP ใบสเปก



FAQ ขั้นตอนงาน บันทึกภายใน



ข้อมูลจากระบบหรือหน้าเว็บที่เราอนุญาต

## ตัวอย่างที่ใช้ทั้งบท

เรามี **คู่มือ 3D Printer 200 หน้า** และอยากให้ AI ตอบคำถามจากคู่มือนี้

# Chunk = ฉีกหนังสือเป็น “บัตรคำ” ที่คั่นง่าย

เราไม่ส่ง PDF 200 หน้าให้ LLM ทุกครั้ง แต่แบ่งเป็นชิ้นเล็ก ๆ



## ตัวอย่าง Chunk

Chunk A: วิธีเปลี่ยนหัว Nozzle

Chunk B: วิธีแก้ First Layer ไม่ติด

Chunk C: วิธีทำความสะอาด Build Plate

# Chunk เล็ก/ใหญ่เกินไป มีผลอย่างไร?

## เล็กเกินไป

เจอคำตรง แต่บริบทขาด เช่น รู้ว่า "220°C" แต่ไม่รู้ว่าจะใช้กับวัสดุอะไร

## พอดี

หนึ่ง Chunk มีใจความครบพอให้ตอบคำถามหนึ่งประเด็น

## ใหญ่เกินไป

ปนหลายเรื่อง คำนวณไม่คม และกิน Context Window มากขึ้น

**เทคนิค:** เริ่มจาก 300–800 tokens แล้วทดสอบกับเอกสารจริง ไม่มีกฎตายตัว

# Overlap = ให้บัตรคำ “เหลื่อมกัน” เล็กน้อย

Chunk 1: ...วิธีเปลี่ยน Nozzle และข้อควรระวัง

Chunk 2: ข้อควรระวัง + การทดสอบหลังเปลี่ยน

เหมือนถ่ายภาพพาโนรามา: แต่ละภาพต้องทับกันนิดหนึ่ง เพื่อไม่ให้ข้อมูลตรงรอยต่อหายไป

# Embedding = “เครื่องสร้างพิกัดความหมาย”

ข้อความเข้า

“รถยนต์”

“automobile”

“กล้วยทอด”

Embedding Model แปลงเป็น

```
รถยนต์ → [ 0.12, -0.44, 0.91, ... ] \nautomobile →  
[ 0.11, -0.42, 0.89, ... ] \nกล้วยทอด → [-0.72,  
0.18, -0.05, ... ]
```

**เปรียบเทียบ:** Embedding เหมือน “เครื่อง GPS ที่อ่านชื่อสถานที่แล้วคำนวณพิกัดให้”

# Vector = “พิกัด” ที่ Embedding สร้างออกมา

Embedding คือ **กระบวนการหา**

**GPS**

Vector คือ **ค่าพิกัดที่ได้**

ในงานจริง Vector อาจมีหลายร้อยหรือหลายพันมิติ จึงไม่ใช่แค่ latitude / longitude 2 ค่า

เปรียบเทียบ

 “สยาม” → Lat/Long

 “วิธีแก้ Nozzle อุดตัน” → Vector

ทั้งสองอย่างคือ **พิกัด** แต่คนละชนิดของแผนที่



# Vector Space = “แผนที่ของความหมาย”



คำที่ความหมายคล้ายกันจะถูกวางไว้ **ใกล้กัน**

รถยนต์ ↔ automobile

**ใกล้**

รถยนต์ ↔ รถ

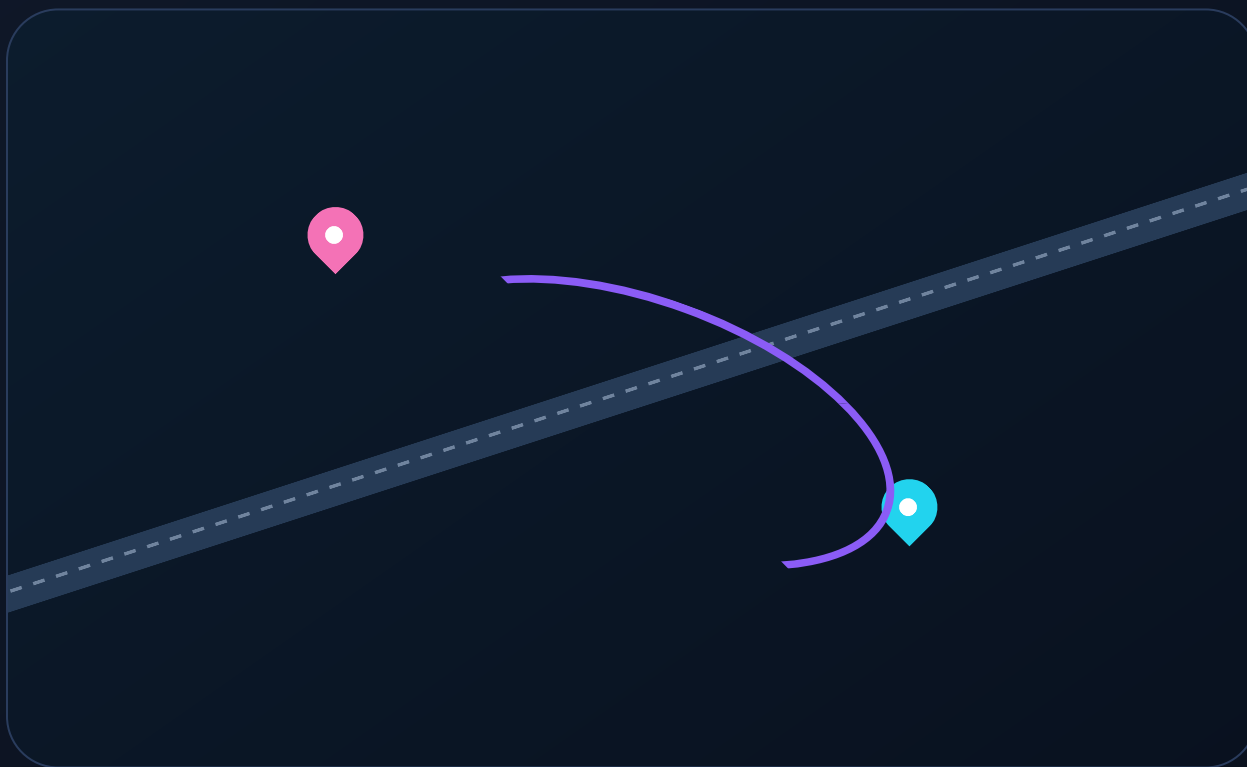
**ใกล้**

รถยนต์ ↔ กล้วยทอด

**ไกล**

ภาพนี้เป็นการย่อให้เหลือ 2 มิติเพื่อสอนแนวคิด

# Vector Database = “GPS Search ของแผนที่ความหมาย”



GPS ไม่ได้แค่เก็บพิกัด — มันช่วย ค้นสถานที่ใกล้เคียง

## Vector DB ก็เหมือนกัน

เก็บ Vector + ข้อความ + Metadata แล้วค้นว่า “Vector ไหนอยู่ใกล้คำถามที่สุด”

[Chroma](#)[FAISS](#)[Qdrant](#)[Pinecone](#)

# Similarity = วัดว่า “พิกัดสองจุดใกล้กันแค่ไหน”

คำถาม

“หัวพิมพ์ตันต้องทำยังไง?”



แปลงเป็น Vector ของคำถาม

Chunk: วิธีแก้ Nozzle อุดตัน

0.94

Chunk: วิธีเปลี่ยน Nozzle

0.81

Chunk: วิธีตั้งค่า Wi-Fi

0.22

ตัวเลขเป็นตัวอย่างเพื่อสอนแนวคิด คะแนนจริงขึ้นอยู่กับวิธีวัดและระบบที่ใช้

# Keyword Search vs Semantic Search

## Keyword Search

มองหาคำที่ตรงกัน

ถาม: "หัวปั๊มพัตต์"

เอกสาร: "Nozzle อุดตัน"

อาจพลาด ถ้าคำไม่ตรง

## Semantic Search

มองหาความหมายที่ใกล้เคียงกัน

ถาม: "หัวปั๊มพัตต์"

เอกสาร: "Nozzle อุดตัน"

มีโอกาสหาเจอ เพราะความหมายใกล้เคียงกัน

# เมื่อผู้ใช้งาน ระบบทำอะไรที่ละขั้น?

## รับคำถาม

"Nozzle อุดตันได้อย่างไร?"

## สร้าง Query Embedding

เปลี่ยนคำถามเป็น Vector

## Vector Search

หา Chunk ที่พิกัดใกล้เคียงที่สุด

## ส่ง Context ให้ LLM

LLM อ่านเฉพาะข้อมูลที่เกี่ยวข้อง แล้วตอบ

# Retrieval = “บรรณารักษ์หยิบหน้าที่เกี่ยวข้อง”

หน้าที่ของ Retrieval ไม่ใช่ตอบคำถาม แต่คือ หา **Context** ที่ดีที่สุด ให้ LLM

RAG เก่งแค่ไหน ขึ้นกับว่า “หยิบข้อมูลถูกขึ้นไหม” มากพอ ๆ กับ ความเก่งของ LLM

## Top-k = 3

- ① วิธีแก้ Nozzle อุดตัน
- ② วิธีถอด Hotend
- ③ การทำความสะอาด Filament Path

# Context = “พอย” ที่เรายื่นให้ LLM ก่อนตอบ

SYSTEM: ตอบจากข้อมูล CONTEXT เท่านั้น\n\nCONTEXT:\n- คู่มือหน้า 42: หาก Nozzle อุดตัน...\n- คู่มือหน้า 43: ก่อนถอด Hotend ให้...\n\nQUESTION:\n\nNozzle อุดตันแก้ยังไง?

## Context ดี

คำตอบมีหลักฐาน

## Context ไม่ตรง

LLM ก็มีโอกาสดูไม่ตรง

## Context เยอะเกินไป

เปลือง Token และอาจรบกวนคำตอบ

# Generation = LLM “อ่านโพยแล้วเรียบเรียงคำตอบ”

Question

+

Retrieved Context

→

LLM

→

Answer

สำคัญ

**RAG ไม่ได้สอนโมเดลใหม่** — เราแค่ส่งข้อมูลที่จำเป็นให้มันอ่าน “ตอนตอบ”



# Prompt ที่ดีช่วย “ลือกกติกา” การตอบ

บทบาท: คุณคือผู้ช่วยตอบคู่มือ 3D Printer\n\nกฎ:\n\n1. ตอบจาก CONTEXT เท่านั้น\n2. ถ้าไม่พบข้อมูล ให้บอกว่าไม่พบ\n3. ห้ามเดา\n4. ระบุหน้าเอกสารเมื่อมีข้อมูล



## Role

ให้รู้ว่ากำลังทำหน้าที่อะไร



## Rules

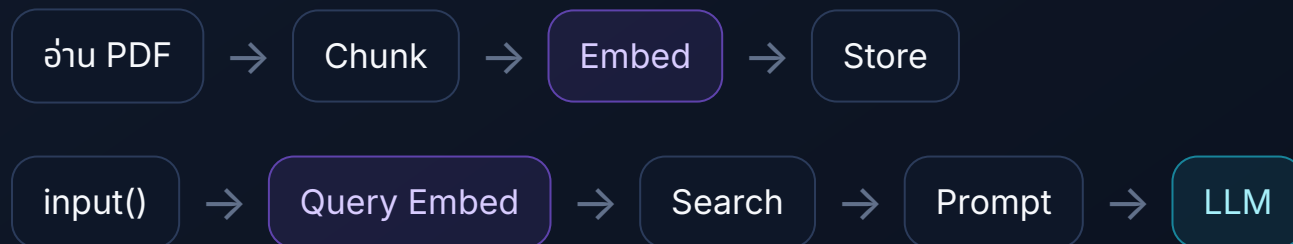
กำหนดว่าจะตอบอย่างไร/ห้ามทำอะไร



## Output

กำหนดรูปแบบคำตอบ เช่น ขึ้นตอน + แล่งอ้างอิง

# Python เป็น “ผู้จัดการ” ที่ต่อทุกชิ้นเข้าด้วยกัน



สำหรับ Beginner: เขียนเป็นไฟล์เดียวให้ Flow ทำงานก่อน แล้วค่อยแยกเป็น modules

# แต่ละ Library รับบทเป็นใคร?



**pypdf**

คนอ่านเอกสาร PDF แล้วดึงข้อความออกมา



**Sentence Transformers / Ollama**

เครื่องสร้าง Embedding หรือเรียกโมเดล



**Chroma / FAISS**

GPS Search ของ Vector



**Ollama / API**

LLM ที่เรียบเรียงคำตอบ



**LangChain**

ตัวช่วยประกอบ Workflow หลายส่วน



**LlamaIndex**

Framework ที่เน้นงานข้อมูล + RAG

# โปรเจกต์แรก: rag.py ไฟล์เดียว

```
# ภาพรวม ไม่ซับซ้อน\n\n# 1. Read PDF\n\n# 2. Split text into chunks\n\n# 3. Embed chunks into vectors\n\n# 4. Store vectors in a database\n\n# 5. User asks a question\n\n# 6. Search for relevant context\n\n# 7. Build a prompt with context and question\n\n# 8. Ask LLM for answer\n\n# 9. Print answer
```

เห็นไหมว่า RAG จริง ๆ คือ **Pipeline** ของงานเล็ก ๆ ต่อกัน

## 3 อย่างนี้ไม่ใช่เรื่องเดียวกัน

### RAG

“เปิดหนังสือก่อนตอบ” เหมาะกับความรู้ที่เปลี่ยนได้  
และเอกสารของเรา

### Fine-tuning

“ฝึกพฤติกรรม/รูปแบบเพิ่ม” ปรับโมเดลด้วย  
ตัวอย่าง ไม่ใช่ฐานข้อมูลค้นเอกสาร

### Agent

“ผู้จัดการที่เลือกเครื่องมือ” อาจเลือก RAG, Web,  
API หรือ Database ตามโจทย์

Hermes Agent



RAG Tool

Web Tool

API Tool

DB Tool

# จำภาพนี้ให้ได้ แล้ว RAG จะไม่งงอีก

 **Document**

หนังสือ

 **Chunk**

บัตรคำ

 **Embedding**

เครื่องสร้างพิกัด

 **Vector**

พิกัดความหมาย

 **Vector Space**

แผนที่ความหมาย

 **Vector DB**

GPS Search

 **Retrieval**

หยิบหน้าที่ใช่

 **LLM**

อ่านแล้วตอบ

**สูตรจำ: ค้นให้เจอ → ส่งข้อมูลที่ใช้ → แล้วค่อยให้ AI ตอบ**